Investigación Persistencia de Datos

Diyer Arley Torres Troya

Carrera de Computación

11 de octubre de 2026

1. Introducción

Una aplicación puede responder lentamente cuando el procesador debe esperar datos. La **latencia de almacenamiento** es el tiempo entre una solicitud y la disponibilidad de los datos; se diferencia del ancho de banda, que mide la cantidad transferida por unidad de tiempo. La **persistencia** permite conservar información después de finalizar un proceso, utilizando medios de almacenamiento más lentos que la RAM [1].

2. Persistencia, jerarquía y fragmentación

La jerarquía de memoria presenta diferencias importantes de latencia (Tabla 1). En un disco duro, el desplazamiento de la cabeza y la espera de rotación influyen en el tiempo de acceso, especialmente en lecturas aleatorias [1].

Medio	Latencia aprox.	Respecto a la RAM
Memoria RAM	≈ 100 ns	$1\times$
SSD NVMe	≈ 80 µs	$\approx 800\times$
Disco duro (HDD)	≈ 8 ms	$\approx 80\,000\times$

Tabla 1: Latencias ilustrativas que varían según el dispositivo.

La **fragmentación interna** desperdicia espacio dentro de los bloques asignados; la **externa** distribuye los datos en ubicaciones no contiguas. Esto puede aumentar las operaciones de acceso, especialmente en discos mecánicos.

3. Impacto en el tiempo de respuesta

El tiempo de una operación puede aproximarse mediante

$$T \approx T_{CPU} + N \cdot L,$$

donde N es el número de accesos al almacenamiento y L su latencia media. Las cachés, el acceso secuencial y las estructuras organizadas por bloques ayudan a reducir las esperas [1]. En los navegadores, mecanismos como `localStorage` permiten persistir datos del usuario [2], [3].

4. Ejemplos de lo aprendido

Ejemplo 1: acceso aleatorio frente a secuencial. Leer 10 000 registros dispersos de 100 bytes en un HDD con una latencia de 8 ms por acceso requiere aproximadamente $10\,000 \times 8\ \text{ms} = 80\ \text{s}$. Si ocupan cerca de 1 MB contiguo, una búsqueda de 8 ms y una transferencia a 100 MB/s tomarían aproximadamente 18 ms, sin contar otros costos.

Ejemplo 2: persistencia con `localStorage`. Este mecanismo almacena cadenas; los objetos se convierten mediante `JSON.stringify` y se recuperan con `JSON.parse`. Sus operaciones síncronas pueden bloquear el hilo principal, por lo que conviene evitar escrituras innecesarias [4].

```
const datos = localStorage.getItem('carrito');
const carrito = datos ? JSON.parse(datos) : [];

function agregar(item) {
 carrito.push(item);
}

function guardar() {
 localStorage.setItem(
  'carrito', JSON.stringify(carrito)
 );
}
```

Código 1: Modificar los datos en memoria y guardarlos cuando sea necesario.

Ejemplo 3: árbol B frente a árbol binario. Para 10^6 registros, un árbol binario balanceado tiene aproximadamente $\log_2(10^6) \approx 20$ niveles. Un árbol B con unas 100 claves por nodo necesita cerca de $\log_{100}(10^6) = 3$ niveles. Si cada nivel implica una lectura de HDD de 8 ms, los costos ilustrativos serían 160 ms y 24 ms, respectivamente [1].

5. Conclusiones

La latencia influye directamente en la velocidad de respuesta del software. El acceso secuencial reduce las esperas frente a las lecturas dispersas. Por ello, reducir los accesos mediante cachés, estructuras por bloques y escrituras oportunas mejora la eficiencia.

Referencias

[1] OpenDSA, "OpenDSA: Interactive Data Structures and Algorithms eTextbook." [En línea]. Disponible en: https://opendsa-server.cs.vt.edu/

[2] M. Newage, "Persistencia de datos en JavaScript: localStorage," *DEV Community*, 2021. [En línea]. Disponible en: https://dev.to/maxwellnewage/persistencia-de-datos-en-javascript-localstorage-419m

[3] "How to store persistent data client-side?" *Stack Overflow*. [En línea]. Disponible en: https://stackoverflow.com/questions/714420/how-to-store-persistent-data-client-side

[4] B. Semah, "Cómo usar LocalStorage en JavaScript," C. Areal, trad., *freeCodeCamp*, 7 de mayo de 2024. [En línea]. Disponible en: https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/